

Leant2: A Unified Architecture Proposal

Lean-native synthesis of programs and their contracts,
consolidated from nine independent design proposals

Prepared for Vladimir Reshetnikov

September 17, 2026

Abstract

Nine independently prepared design proposals for Leant2, a Lean-native successor to the Haskell synthesizers Leant and Djex, were written against the same repository snapshots and the same Lean 4.34.0 toolchain. They agree on more than they disagree. This document consolidates them into one proposal. Its architectural core is the consensus of all nine: exact Lean expressions as the only semantic authority; a transactional search whose unit of backtracking is a partial program together with *all* of its unfinished obligations; recursion admitted only through typed schemas with explicit recursive-call capabilities; behavioral information with graded authority, where tests refute, proofs certify, and only proof-producing analyses may prune; and one acceptance gate that kernel-checks the exact program and the exact contract before anything is published. Where the proposals differ, this document selects the best-developed treatment and records its origin.

Two requirements come from the project owner rather than from the proposals. First, Leant2 has a single adaptive engine: no engine switches and, for now, no user-facing settings. Second, the minimal acceptance baseline is the existing Leant `:synth` golden corpus, thirty transcripts with 278 queries, read semantically: Leant2 must match or exceed every recorded outcome, but may return better candidates in a different order. Nothing here adds new experimental evidence; the evidence sections cite the nine packages and the sibling project Forge, whose kernel-checked certificate checkers and orchestration frontend Leant2 should reuse.

Status. This is a design specification. The nine source proposals each contain a small remotely compiled Lean prototype; collectively they demonstrate the continuation-backtracking core, dependent witness selection, dictionary identity, finite counterexample-guided synthesis, and hand-supplied certified recursion. They do not demonstrate automatic recursion discovery, library indexing, scheduling, or the acceptance gate. Section 15 states exactly what has and has not been shown.

Contents

1	Objective, baseline, and sources	5
1.1	What Leant2 is	5
1.2	How this document was produced	5
1.3	The baseline: Leant’s :synth corpus	6
1.4	What the predecessors established	7
1.5	What to take from Forge	7
2	Principal decisions	8
3	The synthesis problem	10
3.1	Queries and answers	10
3.2	Kinds of constraint	10
3.3	The result algebra	11
3.4	Trust profiles	11
4	Representations and search state	11
4.1	Three representations, one type theory	11
4.2	Origin record and environment epochs	12
4.3	Branch state and the obligation graph	12
5	Transactional search	13
5.1	The atomic attempt	13
5.2	Rule outcomes and blocked states	14
5.3	Joint mode and frozen mode	14
6	Adaptive scheduling	14
6.1	Lanes	14
6.2	Allocation	15
6.3	How the adaptive engine replaces Leant’s settings	15
6.4	Ranking and multiple answers	16
6.5	Resource limits and determinism	16
7	Native construction	17
7.1	Introductions	17
7.2	Application spines	17
7.3	Known-major projections	17
7.4	Polymorphic and higher-rank instantiation	18
7.5	Instances and dictionary identity	18
7.6	Normalization and equality	18
7.7	The constructive propositional lane and classical candidates	18

8	Dependent elimination	19
9	Library composition and carrier invention	19
9.1	The provider index	19
9.2	Component recipes and the join	20
9.3	Providers with contracts	20
9.4	Carrier invention	20
10	Recursion	21
10.1	Routes, in order	21
10.2	Schemas and capabilities	21
10.3	Motive construction	21
10.4	Proof-carrying motives	22
10.5	Accumulator invariants	22
10.6	Termination	22
10.7	The kernel/compiler boundary	23
10.8	Status	23
11	Behavioral constraints	23
11.1	Contract decomposition	23
11.2	Typed observations and residual evaluation	24
11.3	Proof services	24
11.4	Certified counterexample-guided refinement	24
11.5	Reversible observational buckets	25
11.6	Certified pruning	25
12	Caching and equivalence	25
13	The acceptance gate	26
13.1	Pipeline	26
13.2	Receipts	27
13.3	Isolation	27
14	Frontends and deployment	27
14.1	Entry points	27
14.2	Compatibility REPL for the baseline	27
14.3	Worker and controller	28
14.4	Proposed module layout	28
15	Evidence available today	29
16	Evaluation plan	30
16.1	Tiers	30

16.2 Leakage and controls	30
16.3 Metrics and ablations	30
17 Roadmap	31
18 Risks	32
A Provenance ledger	32
B The baseline corpus	34
C Interface sketches	36
References	37

1 Objective, baseline, and sources

1.1 What Leant2 is

Leant and Djex generate implementations for a subset of Lean type signatures, optionally under behavioral constraints. Both are implemented in Haskell, both search over a Haskell-shaped approximation of Lean types, and Djex must remain compatible with its Haskell frontend. Leant2 is the successor that removes all three limitations: it is implemented in Lean, it has only a Lean frontend, and it is designed for Lean types together with behavioral constraints. It cannot decide inhabitation of arbitrary Lean types, and no proposal claims that it can. The target is a large fraction of the types that occur in practice, with every positive answer checked by Lean’s kernel. Three constraints on this document come from the project owner and override anything in the nine source proposals.

Decision 1.1 (Single adaptive engine). Leant2 has one engine. There are no engine switches of the kind Leant exposes (`synth-engine djinn | exference | both`), and no user-facing settings are implemented at first. Budgets, lane weights, verification windows, and candidate counts are internal parameters of an adaptive scheduler (Section 6), derived from the query and from search progress, not from the user. Configuration may be added later behind the same engine, but no acceptance criterion in this document depends on it.

Decision 1.2 (Semantic baseline). Leant2 must pass every `:synth` test in the Leant repository, read semantically: for each recorded query it must match or exceed the recorded outcome. It may return more candidates, better candidates, and candidates in a different order. Section 1.3 makes this precise.

Decision 1.3 (Reuse of Forge). Forge is a sibling project with kernel-checked certificate checkers, an orchestration frontend with full rollback and proof-term auditing, and a written review of the Leant/Djex verification boundary. Leant2 may freely borrow Lean code and ideas from it, and this document does so explicitly (Section 1.5).

1.2 How this document was produced

The nine proposals are numbered P1 to P9 in the order of their directories under `docs/proposals/`. Each is a 32-to-45-page article with a small Lean 4.34.0 prototype checked remotely through the AXLE service, transcribed receipts, and helper scripts. All nine inspected the same Leant revision `6bf05ad` and one of two Djex revisions (`e8778f4`, the standalone head, or `e237e86`, the submodule pinned by Leant); three of them note that the two must not be conflated.

The consolidation rule is simple. An idea present in most proposals is adopted as core and stated once in its clearest form. Where proposals differ in the treatment of one subsystem, the best-developed version is adopted and the alternatives are recorded in Appendix A. Ideas are marked with their origin as ^[P_n] on first use so that the more detailed discussion in the source can be consulted; a marker lists only the proposals that develop the idea most fully, not every proposal that mentions it. Nothing in this document is new evidence. Where the proposals’ prototypes have shown something, Section 15 says which one and how.

Names are unified as follows. What the proposals call the *obligation graph*, *goal forest*, *hole graph*, or *coupled continuation* is called the *obligation graph* here. What they call *recipes*, *search plans*, *construction plans*, or the *control language* is called the *search plan*. What they call *acceptance*, *admission*, *publication*, *finalization*, or the *candidate gate* is called the *acceptance gate*. Their result taxonomies, which range from five to nine constructors, are merged into the single result algebra of Section 3.3.

1.3 The baseline: Leant’s :synth corpus

The Leant repository carries thirty golden-transcript fixtures `test/synth-*.txt`, each piped into `leant -plain` and diffed against `test/synth-*.golden` by `test/run-tests.sh`. Together they contain 278 :synth queries. Of these, 266 produce at least one displayed candidate, five produce the verdict “provably uninhabited”, and seven are deliberate negative controls: three “no term found within the search bounds” under restricted provider settings, two “none passed both Lean verification and the supplied assertion”, one assertion that fails type checking, and one assertion mentioning an unknown name. Twelve queries carry a behavioral assertion in the form `:synth f : T where P`. Appendix B tabulates the fixtures.

The transcripts also contain session declarations (`inductive`, `structure`, `class`, `instance`, `axiom`, `opaque`, `def`, `abbrev`, `universe`), `:reset`, and 170 `:set` lines that switch engines and adjust steps, windows, verification counts, classical mode, library use, and provider discovery. The recorded outputs include exact candidate spellings, ranking order, truncation notes, and engine-specific telemetry.

The goldens cannot be matched textually by a different engine, and Decision 1.1 removes the settings they exercise. The baseline is therefore defined on outcomes, not on text.

Definition 1.4 (Baseline corpus). The baseline corpus is the set of :synth queries in `test/synth-*.txt` at Leant revision `6bf05ad`, each taken together with the session declarations that precede it in its transcript, and each labeled with the outcome recorded in the corresponding golden file. Every `:set` line is ignored. The recorded outcome of a query is one of: *positive* (at least one displayed candidate), *uninhabited* (“provably uninhabited”), *bounded miss* (“no term found within the search bounds”), *rejected* (candidates proposed, none passed), or *preflight error* (assertion rejected before search).

Definition 1.5 (Baseline acceptance). Leant2 passes the baseline when, for every query in the baseline corpus, run under Leant2’s default policy with no per-query configuration and within a fixed per-query wall-clock budget:

1. **Positive.** Leant2 returns at least one *accepted* candidate: kernel-checked at the exact requested type in the exact session context and, for a `where` query, with a kernel-checked proof of the assertion for that exact candidate. The candidate need not be the recorded one, and the order of candidates is unconstrained.
2. **Uninhabited.** Leant2 returns a *refutation* (a kernel-checked proof of $T \rightarrow \text{False}$ in the session context) or a *certified fragment non-derivability* (Section 3.3). For four of the five recorded cases a refutation exists and is preferred; the fifth, `forall p : Prop, p or not p`, is classically provable and must not be reported as refuted.
3. **Bounded miss, rejected.** Leant2 returns either an accepted candidate (this exceeds the recorded outcome and is allowed) or a qualified negative. It must not return an unaccepted candidate as a success.
4. **Preflight error.** Leant2 rejects the query before search with a diagnostic of the same category (ill-typed assertion; unknown name).

The set of accepted candidates is a superset in strength of Leant’s: any candidate Leant2 displays has passed the acceptance gate of Section 13, which is stricter than Leant’s re-elaboration.

Three consequences follow. First, the five `synth-church-rankn`-style fixtures with 69 rank-N and Church-encoding queries are part of the baseline; they exercise nested polymorphic callbacks, universe-polymorphic families, strict-implicit binders, and seeded Church compositions, and are the hardest part of the corpus for a native engine that does not inherit Djex’s rank-N machinery. Second, the `synth-library` fixture requires library composition (`List.map`, `List.foldr`, `List.flatten`, `List.replicate`) from the standard environment, so the provider

index of Section 9 is on the baseline path, not a later milestone. Third, the classical candidates in `synth-manual` (Peirce’s law, double-negation elimination, and the like, produced with `synth-classical on`) count as positive outcomes; Leant2 must be able to offer `Classical.em`-based candidates, labeled as classical, when constructive search returns a certified fragment non-derivability (Section 7.7).

The baseline is a floor. Leant’s further suites also drive `:synth`: the Church behavior ledger (160 cells, 94 with historical acceptance), `test-recursive` (eight recursive-data behavioral scenarios with independent Lean replay), `test-behavioral` (bounded simplification and combined decision controls), and `test-context` (global class-projection discovery). These form the *extended tier* of Section 16: Leant2 should reach them after the baseline, and their historically accepted cells are regression targets, not gates for the first release.

1.4 What the predecessors established

The proposals read the same sources and reach the same conclusions about them.

Leant’s representation boundary. Leant elaborates queries in Lean and rechecks every candidate in Lean, but its search runs in Djex over a serialized fragment (`src/Leant/Synth/Fragment.hs`). The serializer retains structural connectives and selected polymorphic applications but treats term-indexed applications as opaque and describes recursion by bounded templates. This is the boundary Leant2 removes^[all].

Djex’s two traditions. Djinn is a checked propositional prover based on Dyckhoff’s contraction-free calculus LJ_T; it terminates and can report that no derivation exists in its fragment. Exference is a ranked best-first search with class-evidence resolution and resource controls. Djex’s shared infrastructure enforces source ownership, scoped evidence, and independent checking. Leant2 keeps those disciplines without the Haskell compatibility surface^[P6,P8].

Behavioral assertions. At the inspected revision, Leant checks a named assertion against each exact candidate by `decide`, then its negation, then bounded `simp` at both polarities; it distinguishes passed, falsified, and inconclusive, and never treats proof failure as a counterexample. Search produces type inhabitants first and filters afterwards. Every proposal moves the assertion into the search state so that it constrains witnesses before a candidate exists, while retaining the three-way distinction^[all].

The indexing gap. Leant’s current diagnostics locate the remaining integer-indexing failure in bounded composition: a function-valued fold and the integer case step are each found in isolation and missed in composition within the original limits. This motivates the demand-directed component reuse and carrier invention of Section 9. No proposal claims to have solved that fixture^[P1,P6,P7,P9].

The earlier rewrite study. The Leant repository’s own rewrite analysis recommends Lean-hosted semantics, project-matched worker processes, exact source identities, and a kernel validation boundary; those are adopted. It also recommends a pure, neutral search representation with native metavariables confined to preparation and lowering. All nine proposals depart from that point: a neutral representation restricted enough to be easy to search recreates the loss of dependency information the successor is meant to remove. Leant2 searches over native, branch-local dependent constraints, and keeps a serializable plan only for replay, caching, and cross-process transfer (Section 4)^[P1,P6,P9].

1.5 What to take from Forge

Forge [4] is a certificate-producing proof-planning layer above `grind`. It pins the same toolchain as the nine proposals, Lean 4.34.0 with Mathlib 1cf325a0, and has reached a state none of

the Leant2 prototypes has: compiled Lean with proved soundness theorems and a measured comparison. Four parts transfer directly.

1. **The orchestration frontend** (`lean/Forge/Frontend.lean`). The `forge` tactic runs a fixed list of workers, saves the full tactic state before each, restores it on any failure including resource limits, and on apparent success does not take the worker’s word: it instantiates the proof term and rejects it if it contains `sorry`, mentions a free variable outside the goal’s context, fails to type-check against the *original* goal, or depends on an axiom outside the policy, computed with `collectAxioms` over the constants the term uses. It then emits a replay record. This is exactly the proof-service contract of Section 11.3 and the audit half of the acceptance gate of Section 13; Leant2 should start from this code rather than rewrite it.
2. **Certificate checkers with proved soundness** (`lean/Forge/Checker/`). Cone, Farkas, affine-witness, polynomial-recurrence, and conserved-invariant checkers, each with a soundness theorem and an axiom audit that fails the build if a theorem uses an axiom outside `propext` and `Quot.sound`. The design contract `CertificateSpec` (a decoding `denotes`, a decidable `check`, and a proof `sound`) is the shape every proposal wants for its “certified domain plugin” (Section 11.6). Leant2’s behavioral lane should register these checkers as proof services for arithmetic obligations on day one.
3. **Two soundness pitfalls** documented in `docs/IDEAS-FROM-LEANT-DJEX.md` and verified against Lean 4.34.0. `Kernel.Environment.addDecl` skips the kernel when `debug.skipKernelTC` is set in the options the caller passes, so an acceptance step must pin that option off rather than merely not set it. `Lean.addDecl` in `CoreM` commits preliminary constant information immediately and, under `Elab.async`, defers the kernel check to a task, so observing that a declaration is present observes nothing. Acceptance must wait for a positive signal that the checker ran. Section 13 adopts both as invariants.
4. **A checkable fragment refutation**. Forge’s extension round supplies finite Kripke counter-models for intuitionistic propositional logic whose checker recomputes forcing from scratch. This is the object every proposal asks for when it says that LJT non-derivability must not be promoted to a Lean refutation: a certificate that *there is no derivation in the named fragment*, distinct from a proof of $\neg T$. Leant2’s constructive propositional lane should produce this certificate (Section 7.7).

Forge’s status lattice, with its rule that pruning must state its approximation polarity, coincides with the graded-authority principle of Section 11. Its Python search components are research code and are not reused; only the Lean checkers and the frontend are.

2 Principal decisions

The following decisions are held by all nine proposals. Each is stated once here; later sections give the mechanisms.

Decision 2.1 (Exact Lean expressions are the only semantic authority). Targets, contexts, candidates, universe levels, instance dictionaries, and behavioral propositions are Lean `Expr`, `Level`, and local declarations in an identified environment. There is no second type language in which term indices, proof dependencies, or universes become approximations. Indexes, abstract domains, and solver encodings exist, but each is a lossy accelerator with an explicit reconstruction boundary: it proposes, and native unification and the kernel decide^[all].

Decision 2.2 (The unit of backtracking is a partial program with all of its obligations). Solving a hole never commits its value before the remaining obligations that depend on it succeed. An application creates jointly required children; a later proof obligation, index equation, or instance constraint can reopen an earlier data choice. Every prototype implements this as `search`

(`children ++ rest`) inside a saved state, and the canonical failure it prevents is choosing 0 for $\{n : \mathbb{N} // n = 1\}$ and then being unable to prove $0 = 1$ ^[all].

Decision 2.3 (Typing, behavior, and executability are separate results). A term may inhabit the type, satisfy the contract, and compile to acceptable executable code; these are three outcomes, not three spellings of one. A kernel-accepted recursor term can be rejected by the code generator, as several prototypes observed. Executability is a field of the result, not a consequence of typing^[P4,P6,P8,P9].

Decision 2.4 (Recursion enters only through typed schemas). The definition under construction is never an ordinary provider. Recursive calls are available only as *capabilities* that carry the decrease evidence for a specific smaller argument, and a recursion schema records its motive, branches, and termination obligations. One proposal’s first run failed precisely because unfiltered locals exposed a provisional self-reference^[P2].

Decision 2.5 (Behavioral information has graded authority). A finite observation refutes the exact candidate it fails and nothing else. A proof certifies. A heuristic analysis may rank. Only a proof-producing over-approximation may prune a partial program, and only after exhibiting a valid input. Solver verdicts, evaluator timeouts, and failed proof attempts are not evidence^[all].

Decision 2.6 (One acceptance gate). Every positive result crosses one gate that reconstructs the frozen query, closes the program and the proof over the exact telescope, kernel-checks both with kernel bypasses pinned off, audits transitive axioms and runtime dependencies, optionally compiles, and re-elaborates the exported source. Search components cannot construct an accepted result; the constructor is private to the gate^[all].

Decision 2.7 (Negative answers are qualified). “No solution” has at least four distinct meanings: budget exhausted; a named finite grammar exhausted; no derivation in a named fragment, with a checkable certificate; and a Lean proof that no term of the type satisfies the contract. Only the last is an impossibility claim. A negative from a fragment or a budget never enters a cache as impossibility^[all].

Decision 2.8 (Resources are never refunded by rollback). Restoring a metavariable context restores logical state only. Work charged for rule applications, unification, proof attempts, solver calls, and evaluation lives in a monotone ledger outside the snapshot. Cancellation, heartbeat exhaustion, and memory pressure are interruptions that unwind to the controller; they are never caught as “rule not applicable”^[P1,P2,P5,P7,P9].

Together with Decisions 1.1 to 1.3, these fix the shape of the system. Figure 1 shows the pipeline.

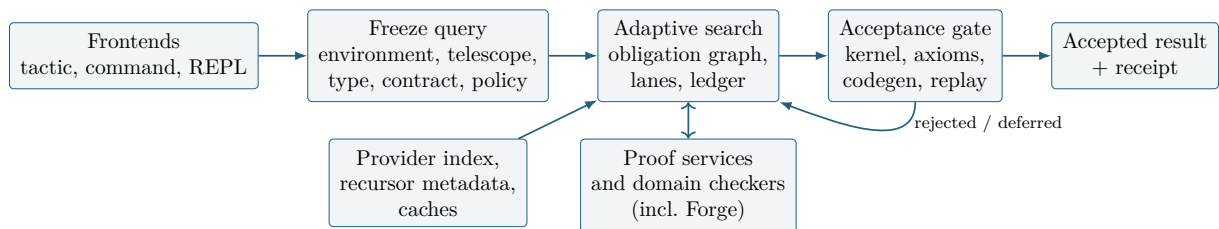


Figure 1: The pipeline. Everything left of the gate is untrusted search; the gate adds nothing to what the kernel checks except refusal.

3 The synthesis problem

3.1 Queries and answers

Definition 3.1 (Frozen query). A frozen query is a tuple

$$Q = (\mathcal{E}, U, \Gamma, T, \Phi, \mathcal{X}, \mathcal{G}, \Pi, \mathcal{B})$$

where $\mathcal{E}$ is the environment with its import identities and revision; U is the universe-parameter context; Γ is the ordered dependent local context, including `let` values and local instances; T is the target with $\mathcal{E}; U; \Gamma \vdash T : \text{Sort } u$; Φ is the optional contract with $\mathcal{E}; U; \Gamma, f:T \vdash \Phi(f) : \text{Prop}$, taken as `True` when absent; $\mathcal{X}$ is a finite set of typed observations; $\mathcal{G}$ is the admitted construction grammar and provider inventory, stated in terms of constants and local terms rather than names; Π is the trust and execution policy; and $\mathcal{B}$ is the resource policy^[all].

The query is elaborated once in the real project environment and frozen before search. Unknown identifiers are errors, not auto-bound implicits; universe parameters remain parameters and are never specialized; instance selection in the statement is part of the frozen question; and no metavariable owned by the caller may be assigned by search except through an explicit permission set returned as an atomic patch^[P5,P6,P8].

Definition 3.2 (Answer). A verified answer to Q is a pair of judgments

$$\mathcal{E}, \Delta; U; \Gamma \vdash t : T \quad \text{and} \quad \mathcal{E}, \Delta; U; \Gamma \vdash p : \Phi(t),$$

where Δ is a finite list of generated auxiliary declarations accepted in dependency order under Π . The two judgments are kept separate rather than packed into a subtype, because T may live in any sort and because a proof that specializes the candidate would describe a different program^[P2,P3,P5,P8].

Proposition 3.3 (Pointwise packaging). *If $\Phi(f) \equiv \forall x:A, R(x, f x)$, then a term $g : \prod_{x:A} \{y : B(x) // R(x, y)\}$ yields $f := \lambda x. (g x).val$ with $\forall x, R(x, f x)$, and conversely, constructively. The engine registers this as a checked transformation and uses the packed form to carry the contract into recursive branches (Section 10); it never pattern-matches on the syntax of Φ to decide correctness^[P1,P3,P9].*

3.2 Kinds of constraint

Four kinds of behavioral input are supplied independently and are never silently converted into one another^[P1,P7]:

- a *hard contract* Φ , which must be proved;
- *observations* $\mathcal{X}$: typed inputs with precondition evidence and expected outputs, or a proposition about particular inputs. They refute candidates and guide search; passing them establishes only themselves;
- *construction restrictions*: grammar predicates such as “exclude `List.reverse`”, enforced on the transitive dependencies of the candidate, not on names;
- *preferences*: ranking signals such as “prefer using every input”, which can never reject a correct program, since some correct programs must ignore inputs.

Leant’s `:synth f : T where P` supplies a hard contract P elaborated under the binder $f:T$; the baseline corpus contains no observations, restrictions, or preferences beyond this.

3.3 The result algebra

The nine proposals propose between five and nine result constructors. They are merged as follows; the internal representation carries orthogonal evidence fields, and the frontend may group constructors for display but the structured API preserves them^[P4,P6,P8,P9].

```
inductive Outcome where
| verified      (t : Accepted) (p : Accepted) -- exact type and exact contract
| typedOnly    (t : Accepted) (pending : Obligations)
| testedOnly   (t : Accepted) (passed : Observations) -- never satisfies a
  universal contract
| refutedCandidate (t : Candidate) (q : Accepted) -- proof of not (Phi t),
  or replayed counterexample
| impossible     (q : Accepted) -- proof of (t : T) -> Phi t -> False, or T ->
  False
| fragmentUnderivable (fragment : Name) (cert : Checked) -- e.g. Kripke
  countermodel for IPC
| grammarExhausted (grammar : Id) (ledger : Incompleteness)
| budgetExhausted (resumable : StateId) (ledger : Incompleteness)
| unsupported     (feature : String)
| infrastructure (error : String)
```

Each accepted object additionally records three independent flags, `kernelChecked`, `compiled`, and `evaluatedOnExamples`, together with the axiom inventory of the proof and the runtime-dependency inventory of the program (Decision 2.3). “Best” candidate means best among verified candidates under a declared cost model; “minimal” requires an exhausted cheaper prefix and is otherwise reported as “lowest cost found under this schedule”^[P6,P8].

The baseline mapping is: Leant’s displayed candidates correspond to `verified` (type-only queries have $\Phi = \text{True}$, so the contract proof is trivial); “provably uninhabited” corresponds to `impossible` or `fragmentUnderivable`; “no term found” corresponds to `budgetExhausted` or `grammarExhausted`; “none passed” corresponds to `refutedCandidate` results with no `verified` result.

3.4 Trust profiles

Three policy profiles are recognized, each fixing an axiom allowlist for proofs and a separate runtime policy for programs^[P1,P3,P6,P9]:

- *strict constructive*: empty axiom inventory beyond explicitly accepted query assumptions; total, computable program; no `Classical.choice` on the computational path;
- *standard*: `propext`, `Quot.sound`, and `Classical.choice` permitted in proofs and erased transports, with actual use always reported; program still total and computable;
- *project-relative*: named project axioms permitted; results certified relative to them.

`sorryAx` is rejected in every profile. A result is never called “axiom-free” when it depends on `propext`; several prototypes’ indexed `cases` did, and the receipts say so. The default profile for the baseline is *standard*, which admits the classical candidates of `synth-manual`, labeled as such.

4 Representations and search state

4.1 Three representations, one type theory

Every proposal converges on the same division of labor, in three tiers^[P3,P5,P8,P9].

1. **Semantic objects.** Native `Expr`, `Level`, names, binder information, local contexts, local instances, and the environment. These are authoritative and are never replaced by their pretty-printed form.
2. **The search plan.** A persistent, serializable record of search actions: introduce this binder; apply this exact provider occurrence with these universe instantiations and this spine prefix; split this scrutinee with this motive; instantiate this recursion schema; ask this proof service with this budget. Plan holes carry a stable identifier, a scope identifier, a native target closed over a dependency-closed telescope, and a role. The plan can be replayed against the frozen query to rebuild native state with fresh metavariable identifiers. It exists for resumption, caching, explanation, parallel transfer, and worker restart; it does not define a competing typing relation.
3. **Accelerators.** Discrimination-tree keys, type-head graphs, abstract domains such as list lengths and affine summaries, finite grammars, and solver encodings. Each is lossy and each declares its reconstruction boundary and its approximation polarity.

Invariant 4.1 (No naked open expressions cross a process boundary). Raw `MVarId` and `FVarId` are never durable. A task shipped to another process or session is a closed plan over an ordered telescope with scope-stable binder positions, exact level and constant names, an environment fingerprint, and a protocol version. The receiver reopens it with fresh identifiers and rechecks; a mismatched fingerprint rejects the package rather than reinterpreting it^[P4,P8,P9].

Within one worker, native snapshots (`Meta.saveState/restore`) are the fast path. A production engine mixes two profiles: hot branches keep a materialized native checkpoint; cold branches are stored as a checkpoint plus the plan suffix needed to rematerialize them, bounding memory^[P5].

4.2 Origin record and environment epochs

The frozen query is stored as an immutable *origin record*: toolchain identity, imported module identities, environment revision, the closed telescope, T , Φ , the component policy, the axiom and execution policy, and the semantic options in force (transparency, active local instances, coercion configuration, grammar restrictions). Fingerprints locate cache entries; they never replace structural revalidation^[P9].

A worker owns an *environment epoch*. Every handle, snapshot, cache entry, provider index, and proof-service reply is stamped with the epoch in which it was made. An undo, a `:reset`, an import change, or a worker restart bumps the epoch and invalidates everything stamped with the old one. Leant’s session model, in which `:reset` and declaration replay change the environment mid-transcript, maps directly onto epochs^[P1,P6,P7].

4.3 Branch state and the obligation graph

Definition 4.2 (Branch state). A branch is $S = (O, M, r, H, D, K, R, \mathcal{X}_S, \rho, c)$: the origin record; the native metavariable and elaboration state; the root program and root contract-proof expressions; the set of open obligations; the dependency graph over them; the delayed-constraint store; the set of recursive-call capabilities in scope; the branch-local observation bank version; the plan suffix that regenerates the branch; and the branch cost vector^[P2,P3,P7,P9].

Each obligation $h \in H$ records its goal metavariable, its local context Γ_h , its target T_h , its residual contract Φ_h if any, and a *role*: program value, type or universe argument, instance evidence, index equality, contract proof, termination proof, invariant, or reconstruction check. Roles guide scheduling; they never affect typing.

The dependency graph has an edge $h_1 \rightarrow h_2$ whenever assigning a metavariable reachable from h_1 can change the type, context, admissibility, or residual contract of h_2 . Reachability

is computed over local declaration types and values, delayed constraints, instance arguments, and the contract, not only over syntactic occurrence; read and write footprints are conservative over-approximations. Obligations that can assign metavariables reachable from one another form *coupled clusters*, the strongly connected components of D , and are scheduled as units. A constructor application whose fields depend on one another is one hyperedge, not a sequence of independent children^[P3,P6,P7,P8].

Invariant 4.3 (Branch ownership). Every unassigned metavariable belongs either to the frozen ambient problem or to exactly one branch lineage. A branch may assign only its own search metavariables; it may not change the frozen statement, specialize the query’s universe parameters, publish declarations, or reuse another branch’s assignment without a checked context embedding. When Leant2 runs inside an unfinished proof, the permitted assignment set for ambient metavariables is computed at freeze time and any delta is returned as one atomic patch^[P6,P7,P8].

Invariant 4.4 (Scope closure, type coherence, dependency coherence, evidence ownership). Every obligation’s target is closed under its own context; an unresolved proof obligation remains an obligation rather than being weakened; assigning a metavariable invalidates any observation, index entry, or summary whose recorded support depends on it; and a proof, counterexample, or summary is meaningful only for the exact expression, context, contract, and policy it was produced for^[P8].

5 Transactional search

5.1 The atomic attempt

The fundamental operation is the atomic attempt: snapshot all state the rule may write, run the rule, recurse on the rule’s new obligations *prepended to the remaining obligations of the branch*, and commit only if that whole continuation succeeds; otherwise restore, retaining only tagged diagnostics and the ledger charges^[all]. The prototype form, common to all nine packages up to naming, is:

```
partial def solve (fuel : Nat) : List MVarId -> SearchM Bool
| []      => pure true
| g :: gs => do
  if fuel = 0 then return false
  if <- g.isAssigned then return <- solve fuel gs
  for rule in rulesFor g do
    let saved <- saveState
    try
      let children <- rule.apply g          -- may assign metavariables shared with
      gs
      if <- solve (fuel - 1) (children ++ gs) then return true
    catch e => if isInterrupt e then throw e -- never swallow cancellation
    restore saved
  return false
```

The production engine replaces recursion by explicit *cursor frames* (focused obligation, siblings, saved state, rule family, next alternative, cost allowance) so that a branch can be suspended, resumed, serialized, and scheduled fairly, but the semantics of the attempt are those above^[P6,P8].

Invariant 5.1 (Commit criterion). A subgoal’s solution may be memoized as an alternative, but its assignments become irrevocable only when every dependent obligation in the enclosing coupled cluster has been discharged or the generalization has been independently justified. An AND-node is a joint obligation with correlated substitutions, not a Cartesian product of independent first answers^[P2].

What a snapshot must cover depends on the rule. A pure **MetaM** rule needs the metavariable context and caches. A rule that elaborates syntax needs the elaborator state, postponed synthetic metavariables, local instances, and diagnostics. A rule that creates declarations needs an environment transaction, and such rules are staged: they run against a scratch environment and are committed only by the acceptance gate^[P2,P5,P9].

5.2 Rule outcomes and blocked states

A rule returns one of **progress** (new obligations, changed holes), **solved**, **blocked**, **rejected** (optionally with a certificate), **unsupported**, or **interrupted**. Three kinds of “blocked” are distinguished and none is a contradiction: blocked on assignments (a delayed equation or instance goal waiting for named metavariables), unsupported by this lane (with the continuation preserved for another lane), and yielded for budget (with the charged work and the continuation)^[P2,P6]. Delayed constraints carry dependency stamps and are woken only when a relevant metavariable is assigned; a flex-rigid equation that becomes a pattern, an instance goal whose output parameters become ground, and a contract that reduces after an assignment are all wake-up events^[P4,P6,P9].

Unification is a service, not a completeness oracle. Its results are classified as deterministic progress, blocked, speculative, or mismatch; a rigid-head mismatch rejects an alternative cheaply; a budget failure or a stuck higher-order equation is unknown and never enters a negative cache^[P3,P7].

5.3 Joint mode and frozen mode

Proof services are invoked in one of two modes, and the modes are distinct APIs^[P1,P2,P7].

- In *joint mode* a proof attempt may assign program holes within the current transaction. Solving $?m = n + 1$ by unification determines the witness. Any such assignment is returned as part of the branch’s construction choice, is subject to the commit criterion, and must respect the executable policy.
- In *frozen mode* the candidate is closed and the service may not replace or specialize it. A proof of $\neg\Phi(t)$ rejects exactly t in exactly this context. A verifier that could mutate its candidate would produce receipts about a different program.

A reopened data choice creates a new candidate identity; evidence transfers between identities only through an explicit checked equality with contract transport^[P4].

6 Adaptive scheduling

Decision 1.1 removes Leant’s engine switches and settings. What replaces them is a single scheduler that runs a *portfolio of lanes* over one obligation graph and adapts its allocation to the query and to progress. This section merges the scheduling designs of the nine proposals into that one engine.

6.1 Lanes

A lane is a policy over the same search actions; lane identity never changes what a candidate means. The initial lanes are^[P1,P2,P7,P9]:

- *exact reuse*: locals, projections of known locals, and direct heads that already have the target type;
- *focused construction*: introductions, constructors, and application spines over locals and the goal’s own constructors;

- *library composition*: indexed retrieval and demand-directed composition of environment providers (Section 9);
- *dependent elimination*: case splits and transports (Section 8);
- *recursion*: schema instantiation, motive generalization, carriers (Section 10);
- *constructive propositional*: a terminating focused calculus over a reified propositional fragment with a certificate for non-derivability (Section 7.7);
- *classical*: `Classical.em` and `Classical.byContradiction` as explicit providers, enabled by policy after the constructive lane has returned a certificate;
- *proof*: the portfolio of Section 11.3;
- *behavioral*: counterexample-guided refinement and domain checkers (Section 11.4);
- *reference*: a conservatively enumerated fallback grammar with increasing cost bounds, which prunes only with a checked certificate.

Every lane exposes a bounded step, `step : Origin -> Task -> Quantum -> WorkerM StepResult`, returning a continuation, proposals, and a work delta. Every generator is a finite-step cursor, not a lazy stream^[P3,P9].

6.2 Allocation

Within a branch, the next obligation is chosen by *pressure*^[P5,P6]:

$$\text{pressure}(h) = a \cdot \text{dependentUses}(h) + b \cdot \text{knownIndexTests}(h) \\ + c \cdot \text{behaviorConstraints}(h) - d \cdot \text{estimatedBranching}(h),$$

with aging to prevent starvation. A cheap proof field that determines a data field is preferred over the data field (proof-first); when no such field exists, witness-first search with continuation backtracking is retained. Coupled clusters are scheduled as units.

Across branches the scheduler runs weighted round-robin over lane families with best-first order within a lane. The priority of a job j is

$$s(j) = g(j) + \lambda_h \hat{h}(j) + \lambda_p \hat{p}(j) + \lambda_d \text{duplication}(j) - \lambda_r \text{relevance}(j),$$

where g is charged cost, $\hat{h}$ an estimate of remaining construction, $\hat{p}$ an estimate of proof debt, and the last two terms penalize syntactic near-duplicates and reward obligations that mention rigid heads and concrete observations. This is a heuristic order, explicitly not an admissible A^* heuristic^[P1,P5].

Fairness is guaranteed structurally, not by weights. The reference lane holds a reserved nonzero share of quanta (one in eight by default) and enumerates admitted plans by a *diagonal schedule* over the tuple (lane, expression-cost bound, recursion-scheme bound, verification allowance), increasing the sum with deterministic ties. Within a tier the heuristic queue is complemented by a reserved FIFO stream so that the fairness claim is honest. Queue caps in fast lanes are permitted, and every dropped state is recorded as an incompleteness event in the ledger rather than treated as a refutation^[P2,P5,P7,P9].

Verification is scheduled alongside generation. A candidate whose contract proof returns unknown keeps a verification continuation with an increasing proof budget in an *unresolved queue*; a bare rejection never shrinks the raw candidate window; a successful-result quota is not a raw-candidate quota^[P7,P8].

6.3 How the adaptive engine replaces Leant’s settings

Leant exposes `synth-steps`, `synth-window`, `synth-verify`, `synth-shown`, `synth-budget`, `synth-timeout`, `synth-classical`, `synth-library`, `synth-providers`, and `synth-engine`. Leant2 has none of them at first. The corresponding behaviors are internal:

- *engine*: the lane portfolio is always active; lane weights adapt to the query shape (propositional goals raise the constructive lane; goals whose head is a library inductive raise library composition; contracts raise the behavioral lane) and to progress (a lane that has yielded nothing in its last k quanta has its weight halved, subject to the reserved share);
- *steps and budget*: the work ledger has a global wall-clock deadline derived from the query’s size and a logical work budget that grows by the diagonal schedule; there is no per-query knob;
- *window, verify, shown*: candidates are streamed to the frontend as they are accepted; the number displayed is a frontend concern; verification is scheduled per candidate as above and never capped by a count;
- *classical*: the classical lane is enabled by the *standard* profile after the constructive lane has produced a certificate, so classical candidates appear later and labeled, which is the behavior Leant’s default configuration approximates;
- *library and providers*: retrieval tiers widen automatically (Section 9); provider discovery is always on, and a result records which tiers were used.

Because allocation adapts, Leant2 may find candidates that Leant’s fixed-window configurations missed (the three “no term found” controls and the two “none passed” verification-window controls of the baseline are examples), and it may rank candidates differently: the cost vector of Section 6.4 replaces Leant’s engine-specific ranking. Both are permitted by Definition 1.5.

6.4 Ranking and multiple answers

The cost of a candidate is a structured vector, not a scalar^[P2,P3,P6,P8]:

(policy violations, term size, eliminations, recursion shape,
transports, nonlocal providers, proof size, runtime estimate).

The first component is an admission constraint. Candidates are ranked lexicographically by a declared profile (small source by default), and a small Pareto frontier of behaviorally distinct candidates is exposed rather than the first result only. Diversity is counted over native terms modulo checked equivalence, not over printed strings; two candidates that agree on all observations are presented as distinct unless a checked equality merges them. “Use every input” is a preference, not a filter.

6.5 Resource limits and determinism

Three independent controls apply: logical work budgets (rule applications, unification steps, normalization steps, proof tasks, solver calls, candidate evaluations), wall-clock deadlines, and process-level limits (memory, output size, external process lifetime). Counters are monotone and charged even after rollback (Decision 2.8). Per-rule limits yield suspended branches; raising a proof budget never regenerates earlier terms. A deterministic mode fixes ordering and seeds so that a transcript replays exactly; a performance mode returns the first result from parallel workers and records which lane produced it^[P3,P4,P9].

Parallelism is restricted to closed request slices: whole lanes, candidate verifications, and connected components of the dependency graph with an independence certificate (disjoint owned metavariables, disjoint synthetic obligations, unchanged shared rigid context). Workers exchange closed plans and certificates, never live metavariable contexts^[P1,P9].

7 Native construction

The construction calculus is bidirectional. Checking $\Gamma \vdash e \Leftarrow T$ drives introductions from the expected type; inference $\Gamma \vdash n \Rightarrow T$ extends heads (locals, constants, projections) by spines and drives eliminations. Rules are labeled as *checked transformation*, *grammar-preserving normalization*, or *heuristic proposal*, and a fairness lane retains the alternatives that aggressive focusing drops^[P3,P6,P9].

7.1 Introductions

For a target that weak-head-normalizes to $\prod_{x:A} B(x)$, introduce a fresh local preserving binder information (explicit, implicit, strict-implicit, instance-implicit). Introduction is existence-preserving for every binder kind, including type parameters, indices, and instance arguments, so it is the default focusing policy for function targets. It does not yield the smallest term: an existing function of the target type must also be considered, so a cheap *exact-term lane* runs before eta-expansion, but a cheap inhabitant never suppresses contract-sensitive alternatives^[P3,P4,P5].

For an inductive target, enumerate all constructors rather than committing to the first; freshen the constructor's universe levels, instantiate parameters, and unify the result with the target in one transaction. An index equation that is inconsistent rejects the constructor; one that is merely blocked keeps it and records a delayed constraint. In dependent pairs and subtypes, the data field remains a choice point until every dependent field and the contract are satisfied; proof irrelevance may deduplicate proofs of a fixed proposition, never data^[P1,P6].

7.2 Application spines

Definition 7.1 (Spine expansion). For a head $c : \prod_{a_1:A_1} \cdots \prod_{a_k:A_k(\vec{a}_{<k})} R(\vec{a})$: freshen the universe parameters of this occurrence; for each admissible prefix length $j \leq k$, in a fresh transaction, create owned holes $?a_1, \dots, ?a_j$ in telescope order with exact binder information, and unify the residual type $\prod_{a_{j+1}} \cdots R$ with the target; keep the full argument vector so that later arguments can constrain earlier ones; classify each hole by role; discharge determined instance and proof arguments cheaply; suspend blocked constraints with dependency stamps; and yield the application together with its whole coupled continuation^[P2,P3,P6,P8,P9].

Partial applications are admitted because a useful application length changes after reduction or when a type variable is instantiated by a function type. Repeated use of one head is allowed. The prototypes build this on `MVarId.apply` with `newGoals := .all` (original argument order, which forces witness-first backtracking to be tested) and `approx := false`; the design treats `apply` as a first adapter and differential-tests any optimized spine planner against it^[P2,P3,P6].

Local providers keep their exact free-variable identities and universes; only global constants are freshened per occurrence. Named universe parameters of the query are rigid; fresh level metavariables of library constants are flexible; there is no implicit universe lifting and no lowering of a polymorphic query to `Type 0`^[P1,P2,P4].

7.3 Known-major projections

Applying `Sigma.snd` to an unknown pair leaves an unknown family and an unknown major premise. Instead, projections are first-class search actions over *known* locals: enumerate `Expr.proj` over local structure values using structure metadata, infer the exact dependent type, and match. This fixed a real regression ($p : \Sigma_n \text{Fin}(n+1) \vdash \text{Fin}(p.1+1)$) generically, and projection-first ordering also fixed a heartbeat exhaustion caused by constructor growth before projections in another prototype^[P1,P5].

7.4 Polymorphic and higher-rank instantiation

Higher-rank arguments are Pi-typed argument holes and are constructed by introduction; there is no rank-N approximation. Type-valued holes are filled by a staged *type-invention ladder*^[P1,P2,P6,P9]:

1. types forced by unification with the rigid target;
2. expected-type-directed construction of polymorphic arguments (introduce the binder rather than enumerate monotypes);
3. a bounded, cost-ordered *type frontier*: types in the telescope, output, and contract; types of locals and of provider domains; small products, sums, sigma and function types over those; registered carrier templates;
4. quantified types and accumulator carriers, with a separate size cost.

Pool exhaustion is a missed instantiation, not impossibility. Stuck higher-order equations are postponed, then bounded-instantiated with small lambda, projection, and application templates, each rechecked by Lean; there is no general higher-order unification. The Church and rank-N fixtures of the baseline are the primary regression set for this ladder.

7.5 Instances and dictionary identity

Two modes must not interfere. In *canonical instance mode*, instance obligations are Lean’s own resolution, delayed until their input parameters are known and woken on assignment. In *explicit dictionary mode*, a dictionary is data: local instances are named providers, the actual dictionary expression is stored at each method occurrence, and search may backtrack over dictionary choices. Two dictionaries of the same class type are never interchangeable without a proved equivalence; only **Prop**-valued classes enjoy proof irrelevance. Cache keys and candidate identity include dictionary payloads. The fixture in which the second of two **Inhabited Nat** values must be selected because the contract says so is a permanent regression^[P1,P3,P5,P6,P8]. Leant2 never manufactures global instances.

7.6 Normalization and equality

Normalization is controlled work: memoized weak-head normalization, a small transparency ladder with per-operation budgets, escalation on failure, and charges that survive rollback. Definitional equality is an operation, not a canonicalizer: there are no pretty-printed normal-form cache keys. The order for an index mismatch such as $\text{Vec } A (n + m)$ against $\text{Vec } A (m + n)$ is cheap definitional equality, then registered proof-producing normalizers, then a bounded equality proof $q : n + m = m + n$ followed by an explicit transport along the equality recursor with a Lean-checked motive; never a raw cast, and never index erasure^[P3,P4,P6,P8].

7.7 The constructive propositional lane and classical candidates

Leant’s baseline includes propositional goals proved constructively (**synth-basic**, **synth-manual**, **synth-prove**) and, with **synth-classical on**, classically. It reports **forall p : Prop, p** or **not p** as “provably uninhabited” because Djinn’s LJT search found no derivation. Leant2 reproduces this capability with two lanes and one rule.

The *constructive propositional lane* reifies the propositional fragment of a goal (connectives $\rightarrow, \wedge, \vee, \neg, \leftrightarrow, \perp, \top$ over atoms that are opaque Lean propositions or types), runs a terminating focused calculus over it, and returns either a Lean proof term or a checkable certificate that no derivation exists in that fragment. The certificate is a finite Kripke countermodel, checked

by recomputing forcing, as in Forge’s extension round [4]. The result is `fragmentUnderivable`, with the fragment named, and it is never promoted to `impossible`^[P1,P4,P6,P8].

The *classical lane* adds `Classical.em`, `Classical.byContradiction`, and `Decidable.em` as explicit providers. It is enabled by the *standard* profile and scheduled after the constructive lane has returned a certificate, so that constructive candidates are found first and classical ones are labeled by their axiom inventory. For the baseline this yields the classical candidates of `synth-manual` and, for `forall p : Prop, p or not p`, both a fragment certificate and a labeled classical candidate.

8 Dependent elimination

Case analysis is a motive-construction problem, delegated to Lean’s own machinery behind a wrapper that returns the branch goals, the old-to-new local mapping, and any transport obligations^[P3,P4,P6,P8,P9].

1. Compute the dependency closure of the scrutinee $x : I \vec{p} \vec{i}$: every later local whose type or value mentions x , its indices, or a selected dependent.
2. Generalize the closure into the motive, preserving order; inspect the real eliminator and its result-sort restriction; build the motive over the indices and the scrutinee.
3. Instantiate each constructor branch with its fields and index equations; reintroduce the dependent context with transformed types; record the branch-to-parent substitution.
4. Discharge impossible constructor and index combinations only by checked evidence: unification contradiction, no-confusion, or a proved local contradiction. Record for each removed branch whether it was removed by kernel-checked contradiction, by a trusted exact-fragment theorem, or by heuristic omission; only the first two support coverage claims.
5. Reconstruct the eliminator application.

Scrutinee selection prefers locals that occur in target indices, that a discriminating contract or observation demands, or that block reduction. Split history is tracked by discriminant identity, so repeated splitting of a recursive input is recognized as bounded unrolling and not as recursion. Elimination restrictions are enforced natively: `False` eliminates anywhere; `Nonempty A` is not an executable `A`; `Exists` is not a computational pair; quotients require a respect proof through their registered lift. Decidable conditionals require an admitted `Decidable` instance and a checked link between the condition and the branch facts^[P2,P4,P7].

Every prototype that eliminated an indexed family with `cases` picked up `propext`; the receipts report it, and the strict profile treats it as a policy decision rather than hiding it.

9 Library composition and carrier invention

9.1 The provider index

A persistent index is built at import over declaration types: identity, universe parameters, telescope, weak-head result shape, argument heads, constructor or projection role, computational-versus-proof classification, and registered specification lemmas. The index is a discrimination tree over normalized result heads with mandatory wildcard and reducible-head buckets; it may over-return but never silently under-returns, and an intentionally narrowed query is never negative evidence^[P4,P5,P8,P9].

Retrieval widens in tiers, each recorded in the result: locals and local dictionaries; the goal’s own constructors and projections; explicitly requested providers; registered project components and

tagged *synthesis lemmas*; nearby namespaces and imports; a budgeted broad scan. A fixed cap such as fifty providers is a heuristic, never “exhaustive”^[P1,P9].

9.2 Component recipes and the join

A component is a recipe ($\Delta \vdash e : \sigma$, contract lemmas, cost, provenance) with Δ the dependency-closed telescope. Reuse under Γ requires a telescope substitution $\rho : \Delta \rightarrow \Gamma$ applied uniformly to e , σ , and the contracts, followed by a native recheck. Forward synthesis stores cheap components in a typed shared DAG; backward synthesis demands result types; a *join* matches demanded premises against stored components and propagates assignments across the whole application^[P6,P7].

A *demand graph* from target holes and observations yields, per demand, a *candidate context slice*: the locals mentioned in the type, contract, and constraints plus a bounded predecessor closure. The slice is searched first and the full context second; slice solutions are transported by explicit weakening and rechecked. This is the direct response to Leant’s indexing diagnostic, in which a handler found standalone is missed in the full context^[P7,P8].

9.3 Providers with contracts

A provider is an exact term plus useful theorems: the declaration, its universe telescope and binder structure, result-head keys, instance dependencies, a use policy, and *specification entries* naming theorems $\forall x, P(x) \rightarrow Q(x, g(x))$ that relate the operation to its inputs and outputs. Entries are checked at registration and never trusted as annotations; a summary must name its justifying theorem and instantiate it at the same occurrence. Multiple specifications per provider are selected by the demanded observation (length, order, multiset). Provider exclusion follows declaration dependencies, not namespace blacklists, so a withheld reference implementation cannot be retrieved through an alias^[P3,P5,P8,P9].

Parametricity is a ranking heuristic and a template source, never a pruning axiom: Lean’s axioms admit nonparametric functions^[P2,P5].

9.4 Carrier invention

When an eliminator or fold is applied and the continuation still consumes an argument after the traversal, the carrier is not the result type. The engine tries, in increasing structural cost, the carrier grammar

$$C ::= R \mid I \rightarrow C \mid C_1 \times C_2 \mid \text{Option } C \mid \Sigma d:D, C(d),$$

seeded by demanded types, with three demand rules: *parameter postponement* (a parameter consumed after the traversal becomes the domain of a function-valued carrier), *result pairing* (two demanded summaries become a product), and *accumulator generalization* (a changing argument becomes an accumulator). Generalization is *behavioral*: it includes arguments whose values change across calls, not only those the type depends on. The worked trace shared by four proposals is defaulted indexing, `List A -> Nat -> Option A`, as `foldr` with carrier $\mathbb{N} \rightarrow \text{Option } A$, $z(i) = \text{none}$, $s(x, r)(0) = \text{some } x$, $s(x, r)(k+1) = r(k)$; one proposal checked a hand-supplied instance of it, none discovered it automatically^[P1,P4,P5,P7,P8,P9].

Base and step are attempted jointly with the outer continuation; failures are memoized only under the full context, motive, and bound. Typed anti-unification of repeated residual obligations proposes shared helper types and let-bound helper holes solved under the intersection of scopes^[P1,P2,P8].

10 Recursion

Recursion is the area where the nine proposals agree most closely on design and have demonstrated the least. Every recursive definition in every prototype was hand-supplied; the tactics filled branch bodies. This section states the consensus design and marks its unimplemented parts.

10.1 Routes, in order

Given a target whose telescope contains an inductive argument, the recursion lane tries, in order^[P2,P3,P4,P5,P8,P9]:

1. a permitted library combinator with a matching specification entry (`map`, `foldr`, `foldl`, option combinators, registered recursors);
2. a nonrecursive eliminator or fold already in the local context, including Church-encoded values with function-valued carriers (Section 9.4);
3. a *structural schema* generated from the recursor metadata of the input type;
4. a strengthened or accumulator-generalized structural schema;
5. a well-founded schema over an explicit measure.

Mutual recursion is a jointly synthesized strongly connected component over a tagged sum with a shared termination argument; nested inductives use their actual recursors; both are later extensions^[P3,P8].

10.2 Schemas and capabilities

Definition 10.1 (Recursion schema). A schema records the function telescope, the decreasing argument, the result family (motive) $M(\vec{i}, x)$, one branch per constructor with its fields and index equations, the set of *recursive-call capabilities* available in each branch, the termination obligations, and the lowering plan^[P4,P7,P9].

Invariant 10.2 (Recursive calls are capabilities). A branch never contains an unrestricted local $f : \forall x, T(x)$. A recursive call is available only as $\text{rec} : \prod_{y:D} \prec(y, x) \rightarrow M(y)$, where the decrease evidence is a constructor field for structural recursion or an obligation $\mu(y) < \mu(x)$ for well-founded recursion. The obligation is synthesized or the call is rejected; the program is never rejected because a measure failed^[all].

The failed first run of one prototype, in which unfiltered implementation-detail locals exposed the provisional definition and Lean rejected the resulting cycles, is the empirical reason for this invariant. The acceptance gate additionally audits the dependency graph of the declaration bundle for provisional self-reference^[P2].

10.3 Motive construction

Definition 10.3 (Motive by dependency-closed generalization). Choose the discriminant $z : D(\vec{p}, \vec{i})$. Partition the surrounding variables into fixed parameters and generalized arguments, closed under dependence and under behavioral change (arguments whose values differ between the call and the recursive call). The candidate motive is

$$M(\vec{i}, z) = \prod \Delta(\vec{i}, z). R'(\vec{i}, z, \Delta),$$

checked by Lean at the recursor's sort. The schema instantiates the recursor, exposes the method holes and induction hypotheses, translates the residual contract per branch, and requires a checked *final specialization* back to the original target^[P2,P5,P6,P9].

Motives are tried on a complexity ladder: the direct result family; generalize the dependent arguments; small subsets of the changing arguments; accumulator and invariant templates. The worked examples common to the proposals are `Vec.map` with $M(n, xs) = \text{Vec } \beta n$, `Vec.zip` with $M(n, xs) = \text{Vec } \beta n \rightarrow \text{Vec } (\alpha \times \beta) n$, vector lookup with $M(n, xs) = \text{Fin } n \rightarrow \alpha$, and accumulator reversal with the motive quantified over the accumulator. Motive-subset search is exponential in the number of changing arguments; the mitigation is the ladder plus *failure-directed generalization*: classify why an induction hypothesis cannot be applied and compute the smallest dependency-closed telescope extension that repairs it^[P2,P6].

10.4 Proof-carrying motives

The central idea for program-and-proof co-synthesis appears in all nine proposals under different names (certificate algebras, refinement motives, certified recursion, packed motives): strengthen the motive so that the induction hypothesis carries correctness^[all].

Proposition 10.4 (Certificate-algebra composition). *Let $C(x) = \{y : B(x) // R(x, y)\}$. A native recursor application with motive C and checked method terms yields $\prod_{x:D} C(x)$, hence an output satisfying R for every input. For function-valued outputs the motive is $C(x) = \prod_{a:S} \{y : B(x, a) // R(x, a, y)\}$. The list instance is the checked theorem*

$$R([], z) \wedge (\forall x xs v, R(xs, v) \rightarrow R(x::xs, s(x, v))) \Rightarrow \forall xs, R(xs, \text{foldr } s \ z \ xs).$$

Each recursive branch receives the smaller output *and* its local specification, so a contract failure backtracks into the branch choice rather than rejecting a finished term. The alternative route, synthesize then prove by induction on the checked equations, is kept as a portfolio alternative and is preferred when the contract is not pointwise. Two proposals checked hand-written instances (`certifiedMap` with a functional `MapRel`; `certifiedAppend`; `indexFold` with its universal correctness)^[P1,P5,P8].

10.5 Accumulator invariants

When a helper with an accumulator is needed, the invariant is a bounded synthesis subproblem, never an assumption. The grammar is built from the public relation, the accumulator variable, the available operations and their registered algebraic summaries, and known unit elements; each candidate yields initiation, preservation, and finalization obligations proved by the proof portfolio. The example shared by six proposals is $\text{go}(xs, acc) = \text{reverse}(xs) ++ acc$ for accumulator reversal; two of them proved the invariant by hand for the fold selected by a finite experiment, and none synthesized it^[P2,P3,P4,P6,P8,P9].

10.6 Termination

The well-founded schema is $F(x) = \text{step}(x, \lambda y (h : y \prec x). F(y))$. Measures come from a finite grammar: natural-number arguments, registered size functions, structural sizes, bounded differences such as $n - i$ with truncated-subtraction semantics handled honestly, lexicographic tuples, and user-supplied relations. Each proposed call yields $\mu(y) < \mu(x)$ as an ordinary obligation; a schema is registered only after Lean’s termination elaboration accepts it. Fuel is legal only when it is part of the interface or proved sufficient^[P1,P3,P5,P8].

10.7 The kernel/compiler boundary

Four prototypes independently produced an axiom-free recursor term that the code generator rejected (“code generator does not support recursor `T.rec` yet”). The consequence is a design rule^[P4,P6,P8,P9]:

Invariant 10.5 (Two products per recursion plan). A recursion schema produces a logical reconstruction, used for proof, and an executable definition, lowered from the plan through the equation compiler with pattern equations and, when needed, `termination_by` and decrease proofs. If the two differ, the engine proves a correspondence theorem $\forall t, f_{\text{eq}}(t) = f_{\text{rec}}(t)$ and checks both, or proves the contract directly for the executable definition. The receipt records `kernelChecked` and `compiled` separately.

Lowering recovers the major argument, patterns, branch locals, capabilities, and generalized arguments from the plan, not from printed recursor text, and elaborates the result in a staging environment that contains only approved dependencies^[P6]. The recursion-schema grammar tags each construction as genuine recursor, bounded unrolling, or nonrecursive, so that repeated case splits are not mistaken for recursion^[P5].

10.8 Status

Demonstrated: filling branch bodies inside hand-supplied structural skeletons on indexed vectors and lists; hand-written proof-carrying motives and their universal correctness theorems; the code-generator boundary. Not demonstrated: automatic schema generation, motive generalization, carrier discovery, invariant synthesis, measure synthesis, lowering. This is the gate that the roadmap of Section 17 treats as the decisive capability milestone^[P8].

11 Behavioral constraints

Behavioral information enters search from three directions: the type constrains shape, the contract propagates backward into holes, and observations and counterexamples constrain choices. None replaces the final proof^[P3,P9].

11.1 Contract decomposition

A *contract decomposer* takes the elaborated contract Φ and a construction step and returns local obligations together with a *reconstruction term* proving that they imply the original. Each extracted view is tagged *necessary*, *sufficient*, or *equivalent*, and the original Φ stays authoritative^[P2,P6,P8,P9]. The initial library of decomposition rules is:

- lambda introduction: a pointwise $\forall x, \text{Post}(x, f\ x)$ becomes $\text{Post}(x, ?r)$ for the body hole (Proposition 3.3);
- constructor application: a record invariant becomes projection equations on the fields;
- decidable conditional: the postcondition splits under the guard and its negation;
- provider application: a registered theorem $\forall a, P(a) \rightarrow Q(g(a))$ turns the demanded Q into the *sufficient* obligation $P(a)$ on the argument. Sufficiency does not license deleting arguments that fail P ; they remain in another lane unless a necessity theorem exists;
- conjunctions, constructor-aligned relations, and registered pre/postconditions.

Multi-call laws stay as higher-order obligations; an opaque higher-order provider stops propagation with a symbolic residual.

11.2 Typed observations and residual evaluation

An observation is a typed substitution into the input telescope, precondition proofs, an exact output observation, and an evidence policy. Dependent outputs are checked at their induced type; **Fin** n values carry their bound; dependent pairs generate the first field before instantiating the second's type; polymorphic functions are tested at several concrete types with distinct dictionary payloads. The default evaluator never runs effects; **I0** is never executed as a test; a timeout is unknown, never false^[P3,P6,P7].

Partial programs are evaluated *residually*: evaluation proceeds until a hole and yields a typed residual (known constructors, closures, suspended applications, conditionals, named holes with observed arguments). A residual is a constraint on a particular use of the hole; multiple observed calls of one hole constrain one shared function, recorded in a hole-observation table keyed by hole and argument tuple so that $h(0) = 0$ and $h(0) = 1$ are recognized as inconsistent. Recursive-call outputs under a sketch are symbolic and never support hard pruning alone^[P2,P6,P7].

11.3 Proof services

Proof obligations are discharged by a portfolio behind one adapter, in cost order: conversion and **rfl**; local assumptions; constructor proofs; **decide** by ordinary kernel reduction; curated **simp** with an explicit lemma set; **omega**; Forge's certificate checkers for cone, Farkas, affine-witness, recurrence, and conserved-invariant obligations [4]; **grind**; Aesop with query-specific rules; optional pinned Mathlib procedures. Induction is a deliberate schema choice coordinated with the recursion plan, not a generic proof step^[P1,P3,P5,P6,P9].

The adapter's contract is Forge's: save the full state before the worker, restore on any failure including resource limits, and on success instantiate the proof term and reject it if it contains **sorry**, mentions a free variable outside the goal's context, does not type-check against the original goal, or depends on an axiom outside the policy. A reply is one of **Proof** (with any permitted assignment patch, in joint mode), **Refutation**, **ReducedGoal** with a reconstruction proof, **Unsupported**, or **Unknown** with the budget consumed. A bare Boolean is not an adequate interface; a failed proof never falsifies a candidate; a result obtained under a branch guard discharges only guarded obligations^[P2,P3,P7]. **native_decide** is not used for proofs; native evaluation may rank or propose but never certifies^[P8,P9].

Proof debt is scheduled: obligations carry cost and readiness, index and arithmetic obligations wake on relevant assignments, and heavy tactics are delayed until the computational holes they mention are assigned or exposed as witness choices^[P1,P5].

11.4 Certified counterexample-guided refinement

Definition 11.1 (Verifier).

$$\text{verify}(t, \Phi) \in \{\text{Proof}(p), \text{Counterexample}(c), \text{Unknown}(r), \text{Failure}(e)\},$$

where c is a value of the actual input type with proofs of its preconditions and a checked violation. A counterexample enters the bank only after replay in Lean's exact interpretation; a solver model is a proposal until decoded and replayed; an evaluator timeout adds nothing^[P1,P4,P5,P8].

The loop: choose the next candidate consistent with the bank; try to prove $\Phi(t)$; on unknown, schedule a counterexample search and keep the candidate in the unresolved queue at a larger proof tier; on a certified counterexample, add it to the bank, repartition the observation buckets, and wake suspended work; publish only through the acceptance gate. "Failure of one is not success of the other": a proof attempt and a counterexample search for the same candidate run as separate resumable tasks^[P6,P7,P9].

Proposition 11.2 (Progress of finite counterexample search). *With N candidates in a finite grammar, permanently retained valid counterexamples, and bank-consistent selection, at most N rejections occur; unknown verdicts do not count as rejections^[P3,P8].*

Finite domains are promoted to universal statements by an untrusted enumerator that proposes a literal and a kernel `decide` that proves the universal statement, decoupling synthesis time from proof size; the finite promotion theorem $(\forall x, x \in L) \wedge (\forall x \in L, R(x)) \Rightarrow \forall x, R(x)$ is checked once^[P6,P9]. Two prototypes implemented finite Boolean CEGIS in Lean with the universal correctness proved by ordinary `decide`; one implemented a proof-carrying certifier `certify : Affine → Option {c // Spec c.eval}` whose acceptance object is a Lean proof rather than a Boolean^[P7,P8,P9].

11.5 Reversible observational buckets

Candidates are grouped by their observation vector $\text{obs}_{\mathcal{X}}(t) = (t(x_1), \dots, t(x_m))$. A bucket holds an active representative *and* its members or their regeneration recipes; a new observation splits buckets and reactivates members; a second liveness rule rotates parked members even without new observations, since a parked candidate may have a shorter proof. Permanent merging is unsound: the Boolean pair (constant true, identity) agrees on `true` and differs on `false`, and three separate finite experiments showed that permanent deduplication on the initial bank loses the correct program (identity versus reverse; x versus y on $(0,0)$). Observational agreement is provisional and never a cache equivalence^[P1,P2,P3,P5,P9].

11.6 Certified pruning

Proposition 11.3 (Sound sketch pruning). *Let P be a partial program with completions $\text{Comp}(P)$ and let $A(P)$ be an abstract summary with $\{\text{Beh}(t) : t \in \text{Comp}(P)\} \subseteq \gamma(A(P))$. If some valid input ρ_0 has a defined observation for every completion, and a checked necessary condition K on that observation satisfies $\gamma(A(P)) \cap K = \emptyset$, then no completion of P satisfies Φ ^[P3,P5,P6,P8,P9].*

A domain plugin implements Forge’s `CertificateSpec` shape: a typed reification, an interpretation into Lean, a soundness theorem for its transfer rules, and refusal cases. Its replies are `Unsupported`, `RankingHint` (no logical authority), `NecessaryConstraint` with proof, `Counterexample` with validity evidence, or `NoCompletion` with a certificate and its exact grammar scope. Missing transfer rules yield top, never bottom; an under-approximation may produce witnesses but may never justify pruning; grammar identity is part of every certificate’s applicability. Initial domains are list lengths and sizes, affine and linear arithmetic (Forge’s checkers), constructor tags, bounded machine integers, and small multisets. One proposal proved the length-abstraction necessity and sufficiency for its fold grammar; another proved an exact five-list separator family for affine folds and generated twenty-seven universal contracts from it^[P3,P4,P5,P7].

External solvers have three distinct uses, each with its own validation: proposing sketch choices, proposing counterexample inputs, and proving theory obligations by a reconstructed certificate. `unsat` on an under-approximate encoding excludes only that encoding. Typed SMT sketches encode dependent holes either *branch-expanded* (each witness alternative owns a specialized child grammar) or *native-first* (the solver picks the witness and Lean generates the residual); untyped multiplexers are not used^[P2,P5,P6].

12 Caching and equivalence

Three equivalences are kept apart: structural identity, checked semantic equality, and finite observational agreement^[P8].

Cache keys include the environment epoch and provider policy, the canonical dependent context preserving `let` values and instance identities, the target, the residual contract, the transparency setting, universe parameters, the assignment state of shared metavariables, the grammar identity, domain versions, and the observation-bank version; never a pretty-printed type. Positive entries are closed term templates over a canonical telescope, reinstantiated through an explicit context embedding and rechecked. Negative entries must state whether they are a semantic refutation, a bounded-grammar exhaustion with its bound and provider set, or an operational miss; a failure at fuel twenty says nothing about fuel one hundred; a timeout is never cached as uninhabitability^[P1,P2,P4,P5,P7,P9].

Nogoods store the smallest justified dependency slice, tagged with request, contract, instance selections, epoch, and grammar scope; a budget exhaustion is a penalty, not a nogood. Definitional-equality comparisons must not mutate branch unknowns. E-graphs are permitted only over a reified typed fragment with proof-carrying edges; proof irrelevance merges proofs, never dictionaries. Library learning stores successful closed subterms as checked helper schemas, with leakage rules for evaluation^[P1,P2,P3].

13 The acceptance gate

13.1 Pipeline

The gate is the only constructor of an accepted result. It runs, in a fresh native state^[all]:

1. **Restore the origin.** Check the environment epoch and the origin record; reject stale generations. Reconstruct the exact type and contract from the frozen query, never from the candidate’s inferred type.
2. **Close.** Instantiate all expression and level metavariables; reject any remaining hole, dangling free variable, loose bound variable, or unresolved elaboration obligation. Close the program and the proof over the original telescope, preserving `let` values; generalize only permitted universes and locals.
3. **Bundle.** Collect auxiliary declarations in dependency order; check that no declaration references a provisional self-reference and that every helper body is present. The published theorem refers to the published definition by its exact generated name.
4. **Kernel-check.** Add the bundle through the ordinary checked declaration path in a scratch environment, with `debug.skipKernelTC` pinned to `false` in the options passed, and with a positive signal that the kernel task completed rather than mere presence of the constant (Section 1.5). Check the program at the frozen T and the proof at Φ applied to *that* program, retained even when Φ ignores its argument.
5. **Audit.** Compute the transitive axiom closure of the proof and the transitive runtime-dependency closure of the program separately; enforce the profile (Section 3.4), rejecting `sorryAx`, unapproved axioms, `implemented_by` and `extern` outside the runtime policy, `partial` and `unsafe` dependencies, and any use of the withheld reference implementation. Policies are over the dependency graph, not over strings.
6. **Compile.** When the execution policy requires it, compile the exact program and record the outcome; a kernel-accepted term that fails code generation is not `compiled`.
7. **Replay the source.** Delaborate the result in the pinned presentation context, re-elaborate the source against the frozen signature, and compare meaning with the accepted expression; on mismatch report a “newly checked source variant” rather than the original. Text is never the authority: implicit arguments, notation, and instance drift change on re-elaboration.
8. **Publish atomically** with a receipt. Nothing is published on any failure.

Theorem 13.1 (Conditional acceptance soundness). *Assume the validation environment is the recorded one, the kernel and the imported environment are correct, and the gate accepts only*

*closed instances of the frozen contract. Then every published **verified** result satisfies the two judgments of Definition 3.2 relative to the displayed premises and the allowed axioms. No search step, heuristic, index, solver, or proof service occurs as a premise. This is an architectural implication, not a formal verification of the gate^[all].*

Invariant 13.2 (Inference is not acceptance). Success of `inferType`, `isDefEq`, `Meta.check`, or a tactic’s “no goals” is never acceptance. Forge’s test workers demonstrate each failure: a goal assigned `True.intro` without a type check, a proof mentioning a phantom local, an admitted theorem, and a `Classical.choice` proof under a constructive policy [4].

13.2 Receipts

A receipt records the request identity; toolchain, import, and environment identities; provider and policy selections and the retrieval tiers used; the accepted declarations and the contract type; the axiom closure of the proof and the runtime-dependency closure of the program; the flags `kernelChecked`, `compiled`, `evaluatedOnExamples`; the source-replay outcome; search bounds and the incompleteness ledger; and the accounting of rejections, inapplicabilities, and infrastructure failures. Hashes are identity checks, not semantic equivalence. Remote-checker receipts, such as the AXLE transcriptions in the nine packages, are operational evidence and are never confused with local kernel artifacts^[P3,P7,P8].

13.3 Isolation

In-process private constructors protect against mistakes, not against hostile metaprograms. The strict deployment profile runs the gate in a separate process with the same kernel against the project snapshot, receiving only the serialized bundle and never executing proposed tactic code. Untrusted extensions (learned proposers, external solvers, plugin metaprograms) run in a sandboxed proposal worker with a restricted publication interface. An independent checker such as Lean4Lean [25] is an optional higher-assurance route, reported only when performed^[P3,P6,P9].

14 Frontends and deployment

14.1 Entry points

All entry points build the same frozen query and share the gate^[P3,P6,P9]:

- a tactic, by `leant2`, which fills the current goal; when the goal is data, it may be combined with a contract as a subtype or with a separately supplied `satisfying` clause;
- a command, `##synth T satisfying fun f => P using [providers]`, for exploration without registering declarations;
- a declaration command, `synth_def name : T satisfying fun f => P`, which publishes the implementation and its correctness theorem atomically, at command-elaboration level so that recursive declaration bundles and the compiler are available;
- a batch API used by the worker and by test harnesses.

Unknown names in a contract are errors, not auto-bound implicits. The proposed syntax is a design sketch; none of the nine prototypes implements it.

14.2 Compatibility REPL for the baseline

To run the baseline corpus, Leant2 ships a small REPL adapter that accepts Leant’s transcript syntax: session declarations, `:synth T`, `:synth f : T where P`, `:reset`, `:quit`, and

##eval/##check. Every `:set` line is accepted and ignored (Decision 1.1). The adapter is an adapter, not a second elaborator: it feeds declarations to a session environment, bumps the epoch on `:reset`, and submits each query to the batch API. Its output is a new golden format that prints outcomes and accepted candidates with their axiom labels, and the baseline harness compares outcomes by Definition 1.5 rather than by text. Leant’s session-binding convention (`it1`, `it2`, ...) is preserved so that transcripts such as `synth-manual`, which evaluate `it2` after a query, replay.

14.3 Worker and controller

The core is a Lean library compiled with the project’s own toolchain; a globally compiled executable must not load incompatible `.olean` files. A lightweight controller, also in Lean, owns sessions, progress, cancellation, and durable requests; a replaceable project-local worker owns the environment, local contexts, metavariables, caches, and search state, and is launched under the project’s Lake configuration. Heavy solver calls and untrusted runtime evaluation go to subordinate processes. Protocol messages carry a version, the query identity, the epoch, and a cancellation token; replies carry request and frame identifiers so that a stale reply cannot satisfy a newer obligation. Durable state is source commands, the origin record, and plans; never pointers or metavariable identifiers. There is no Haskell component^[P1,P2,P6,P7,P9].

The narrow, version-sensitive adapter over Lean’s metaprogramming API is isolated in one module with toolchain-pinned regression tests for success, failure, rollback, and interruption of each operation. The nine prototypes exercised exactly this surface on Lean 4.34.0^[P1,P4,P6,P9]:

```
Meta.saveState, Meta.restore, MVarId.intro1P, MVarId.apply, MVarId.cases,
MVarId.induction, mkConstWithFreshMVarLevels, whnf, Meta.check, addDecl,
collectAxioms.
```

14.4 Proposed module layout

Module	Responsibility
<code>Leant2.Frontend</code>	tactic, commands, REPL adapter; builds frozen queries
<code>Leant2.Core.Origin</code>	origin record, epochs, policies, result algebra
<code>Leant2.Core.Plan</code>	serializable search plan, replay, cross-process packaging
<code>Leant2.Native</code>	the version-sensitive adapter and transaction combinators
<code>Leant2.Search</code>	obligation graph, cursor frames, scheduler, ledger
<code>Leant2.Rules</code>	introductions, spines, projections, constructors, transport
<code>Leant2.Elim</code>	dependent elimination and motive construction
<code>Leant2.Index</code>	provider index, tiers, component recipes, demand graph
<code>Leant2.Recursion</code>	schemas, capabilities, motives, carriers, lowering
<code>Leant2.Contract</code>	decomposer, observations, residual evaluation, buckets
<code>Leant2.Proof</code>	proof-service adapter; Forge checkers, <code>grind</code> , Aesop
<code>Leant2.Domain</code>	certified abstract domains and solver bridges
<code>Leant2.Accept</code>	the gate, audits, source replay, receipts
<code>Leant2.Worker</code>	controller, worker, isolation

Core modules depend only on Lean and its standard library. Mathlib-backed proof methods, Forge’s Mathlib-dependent checkers, SMT adapters, and learned ranking live in optional packages pinned separately^[P8].

15 Evidence available today

Every one of the nine packages compiled its Lean sources remotely through the AXLE service on Lean 4.34.0, with `import Lean` only, `ignore_imports=false`, and `theorems_only=false`, invoked from Wolfram Language. No package had a local Lean toolchain; no package ran an independent checker; every receipt is a transcribed summary of the service response, not a raw capture. Whole-file service times of roughly 1.2 to 2.1 seconds are elaboration latencies and say nothing about synthesis cost. Table 1 summarizes the prototypes.

P	Tactic / files	Synthesized (auto)	Hand-supplied and notable
1	<code>leant2_core</code>	21 examples, 4 controls; 24/25 axiom-free	<code>certifiedMap</code> , <code>affineFold</code> ; Python bucket and 81-sketch CEGIS
2	<code>leant2_core</code>	12 declarations; 10 axiom-free	branch bodies in <code>vectorMap</code> skeleton; <code>ReverseCertificate</code> ; 471-term Python CEGIS
3	<code>micro_synth</code>	15 declarations; all axiom-free	2 elimination skeletons; 3,873-term Python CEGIS with length abstraction; handwritten certificates
4	<code>leant2_core</code>	14 definitions; 19/20 axiom-free	<code>vecMap</code> template; 27 affine contracts generated and checked with Mathlib; 4 negative-control files
5	<code>leant2_mini</code>	11 definitions; 13/14 axiom-free	<code>vectorReplicate</code> branches; <code>certifiedAppend</code> ; <code>prefixCannotComplete</code> ; 1,875-term Python lab
6	<code>leant2_pilot</code>	14 declarations; all axiom-free	<code>Vec.map</code> , <code>Vec.zip</code> , <code>reverseAux</code> ; <code>safePruneAt</code> ; recursor codegen probe
7	<code>solve</code>	13 declarations; 19/20 axiom-free	<code>vecMap</code> skeleton; affine CEGIS in Lean with proof-carrying certifier
8	<code>leant2_core</code>	11 definitions; 12/13 axiom-free	Boolean CEGIS in Lean (208 trees, all 16 tables); <code>CertifiedFold</code> with <code>indexFold</code>
9	<code>leant2_demo</code>	24 audits; 23 axiom-free	<code>listMap</code> with supplied spine; Boolean CEGIS in Lean (9,168 trees)

Table 1: Prototype evidence in the nine packages. “Axiom-free” counts printed empty inventories; the remainder use `propext`, and in two certificate files also `Quot.sound`. No inventory contains `sorryAx`.

Collectively the prototypes demonstrate:

- the continuation-backtracking core of Decision 2.2, with witness-first regressions such as $\{n // n = 1\}$, $\{f // f \text{ 11 } 29 = 29\}$, and $\{c : \text{Choice} // \text{Good } c\}$ that no first-answer-then-prove pipeline passes;
- dependent application, dependent pairs, indexed vector head and tail by `cases`, transport along a hypothesis, higher-rank arguments, independent universe parameters, strict-implicit binders;
- dictionary identity: selecting the second of two same-typed dictionaries because the contract says so;
- finite counterexample-guided synthesis inside Lean with universal correctness by `decide`, and a proof-carrying certifier;
- hand-written proof-carrying motives and their universal correctness theorems;
- the kernel/compiler boundary, observed four times;
- that the checking service’s `okay` flag is insufficient: a custom axiom and a `sorry` both returned `okay=true` and were caught only by the failed-declaration report and the axiom audit.

They do not demonstrate: automatic recursion, motive, carrier, or invariant discovery; a provider index or any library composition beyond explicitly named providers; scheduling, fairness, or resource accounting beyond depth fuel; caching; serialization of plans; the acceptance gate as a

separate component; any run of the Leant baseline corpus; any comparison with Leant or Djex; the original integer-indexing and two-universe fixtures. Every proposal says so.

16 Evaluation plan

16.1 Tiers

1. **Baseline tier.** The corpus of Definition 1.4, judged by Definition 1.5. This is the release gate for the first usable Leant2 (G3 below). It is run through the compatibility REPL of Section 14.2 in deterministic mode, and its results are recorded as a new golden set that lists outcomes, accepted candidates, and axiom labels.
2. **Extended tier.** Leant’s Church behavior ledger (the 94 historically accepted cells are regression targets), `test-recursive`, `test-behavioral`, and `test-context`, each re-expressed as batch requests with the same session declarations and with independent Lean replay of every accepted candidate.
3. **New corpus.** A versioned corpus of exact requests, not type strings: type, context, universes, contract, imports, admitted implementation components and proof-only components, permitted axioms, profile, and any supplied sketch. Strata: polymorphic and higher-order plumbing; dependent pairs and records; indexed inductives and transport; local dictionary identity; library composition; structural recursion; accumulator algorithms; well-founded algorithms; behavioral contracts (finite examples and universal laws separately); impossible and adversarial requests in a separate denominator. Held-out families, not held-out rows. Three assistance levels: no sketch, supplied recursion skeleton, supplied invariant sketch^[P4,P6,P9].

16.2 Leakage and controls

The reference implementation, its aliases, its equation lemmas, and any oracle are excluded from the provider inventory by transitive dependency audit; the search worker and the reference oracle run in separate processes; library reuse, composition, and new recursion are reported separately. A permanent adversarial acceptance suite includes: a proof about a different term; two `Inhabited Nat` dictionaries with different payloads; `Vec a n` offered for `Vec a m`; scope escape; a candidate that passes every current example and fails the next; a verifier timeout; an invalid decrease; a kernel-accepted but uncompileable term; a hidden reference implementation; an unapproved axiom; a printer that changes an implicit argument; a worker death mid-request; a stale cache after `:reset`; a callback assigning an ambient metavariable; a falsely strengthened goal^[P2,P5,P7,P8].

16.3 Metrics and ablations

Metrics are reported as distributions with censored timeouts: fraction of requests with a verified implementation under budget; time to first typed, tested, and verified candidate; charged work by kind; kernel and replay cost; peak memory; candidate and proof size; axiom and runtime-dependency profile; counterexample replays; reactivated bucket members; and the failure category distribution. Ablations disable one mechanism at a time: continuation backtracking (replaced by first-answer commitment), known-major projections, proof-first ordering, result-head indexing, context slices, contract propagation (replaced by post-filtering), proof-carrying motives, carrier discovery, certified pruning, reversible buckets, the reference lane, and the adaptive lane weights (replaced by fixed Leant-like windows)^[P1,P2,P4,P5].

A comparison with Leant and Djex is meaningful only on the shared fragment with pinned repositories, toolchains, and limits; the baseline corpus is that fragment, and the comparison is per-query outcome and time, not an aggregate percentage.

17 Roadmap

Gates are evidence, not dates. Each gate names its exit criteria; nothing in a later gate is a prerequisite of the baseline.

G0 Trusted path.

Origin record, epochs, the native adapter with rollback and interruption tests, the acceptance gate with Forge’s audit, receipts, source replay. Exit: malformed, admitted, scope-escaped, wrong-target, stale-epoch, and forbidden-axiom proposals are refused; a false contract is refused; a closed accepted result replays in a fresh module without synthesis tactics.

G1 Native core.

Obligation graph, cursor frames, atomic attempts, introductions, spines with prefixes, known-major projections, constructors, exact locals, dictionary modes, the constructive propositional lane with its certificate, the classical lane, transport. Exit: the witness-first regressions; independent universes; strict implicits; two dictionaries with different payloads; `synth-basic`, `synth-manual`, `synth-inductives`, `synth-prove`, and `synth-instance-implicit` pass the baseline criterion.

G2 Composition.

Provider index with tiers, component recipes and the join, demand graph and context slices, the adaptive scheduler with the reserved reference lane, caches with epoch invalidation, the type-invention ladder. Exit: `synth-library`, all `synth-provider-*`, `synth-djinn-providers`, `synth-quantified-provider`, and all rank-N and Church fixtures pass; no starvation under the fairness test; cache invalidation after `:reset`.

G3 Baseline release.

Contract propagation, typed observations, joint and frozen proof modes, the proof portfolio including Forge’s checkers, certified CEGIS, reversible buckets, the compatibility REPL. Exit: **the full baseline of Definition 1.5 passes in deterministic mode**, including `synth-behavior`; the adversarial acceptance suite passes; per-query time is reported against Leant’s under equal wall-clock budgets.

G4 Dependent recursion.

Structural schemas from recursor metadata, capabilities, dependency-closed and behavioral motive generalization, proof-carrying motives, lowering through the equation compiler with the correspondence route. Exit: `map`, `append`, `length`, `zip`, `index-safe access`, and `filter` with relational contracts, with no handwritten skeleton; the two-products invariant on every recursive result.

G5 Certified behavior and carriers.

Contract decomposer with reconstruction terms, certified domain plugins on the `CertificateSpec` shape, function-valued carrier discovery, accumulator invariant synthesis, measure synthesis, well-founded schemas. Exit: defaulted indexing discovered without a sketch (the predecessors’ open fixture); accumulator reversal with a synthesized invariant; a bare solver verdict fails certification; the extended tier reaches its historically accepted cells.

G6 Scale.

Persistent indexes, plan serialization and worker restart, parallel lanes with independence certificates, mutual and nested recursion, effect adapters, optional SMT and learned ranking, project-scale evaluation on the new corpus.

18 Risks

- **State coupling.** The cost of `saveState/restore` per attempt, heartbeat accounting across branches, and memory of retained frontiers are unmeasured. Mitigation: cursor frames, the two memory profiles, profiling at G1 before the scheduler is tuned^[P5,P9].
- **Rank-N coverage.** Leant inherits Djex’s rank-N and Church machinery; Leant2 replaces it with native introduction plus the type-invention ladder. The 69-query Church fixture is the test, and it sits in G2, before the baseline release.
- **Recursion discovery.** Motive generalization and carrier invention are described as bounded ladders without cost data. They are deferred to G4 and G5 so that the baseline does not depend on them.
- **Proof bottlenecks.** Universal contracts need proofs the portfolio may not find. Mitigation: proof-carrying motives, Forge’s checkers for arithmetic, the unresolved queue with growing budgets, and honest `typedOnly` and `testedOnly` outcomes.
- **Evidence provenance.** All current evidence is remote and transcribed. The first local action of G0 is to replay the nine prototype files under a local Lean 4.34.0 toolchain and record raw receipts.
- **Scope creep.** The proposals together describe more than any team can build; the gates above are ordered so that a usable, baseline-passing Leant2 exists at G3 with recursion, carriers, and scale still open.

A Provenance ledger

Each row names an idea adopted in this document, the proposals that develop it most fully, and the alternatives that were not adopted. “All” means every proposal contains the idea in some form.

Idea	Adopted from	Alternatives and notes
Exact Expr as semantic authority; three representations	all; P3, P5, P8, P9	The earlier rewrite study’s neutral IR was rejected by all nine. P1 keeps only replay plans serializable; P6 names the pair “search plan / constraint state”.
Whole-continuation backtracking	all	P2 adds the commit criterion; P3 names the continuation ownership invariant; P1 contrasts with Aesop safe rules.
Separate program and proof judgments	P2, P3, P5, P8	P4 uses a subtype in experiments and separates in the interface; P9 packs with PSigma when needed.
Pointwise packaging	P1, P3, P9	Registered as a checked transformation, not syntactic.
Four constraint kinds	P1, P7	P1 names hard contract, example, preference; P7 adds construction restriction and operational objective.
Result algebra	P4, P6, P8, P9	Merged from taxonomies of 5 (P1) to 9 (P4, P9) constructors; fragmentUnderivable from P5; three flags from P9.
Trust profiles	P1, P3, P6, P9	Names vary; all separate proof axioms from runtime dependencies.
Origin record and epochs	P9; P1, P6, P7	“Environment generation” (P1, P3), “epoch” (P6, P7).
Obligation graph with roles, coupled clusters	P3, P6, P7, P8	P7 makes constructor application a hyperedge; P8 states four invariants.
Branch ownership	P6, P7, P8	P1 calls it the ownership barrier.
Three blocked states	P2, P6	
Joint versus frozen proof modes	P1, P2, P7	P2’s proof-service firewall; P4’s candidate identity on reopening.
Lane portfolio with reserved reference lane	P1, P2, P7, P9	P5 fixes 1 in 8 quanta; P9 the diagonal tuple schedule and FIFO stream.

Idea	Adopted from	Alternatives and notes
Pressure and priority formulas	P5, P6; P1	Weights are placeholders to be tuned at G1.
Cost vector and Pareto frontier	P2, P3, P6, P8	
Cursor frames	P6, P8	
Bidirectional focusing, rule labels	P3, P6, P9	
Spine prefixes	P2, P3, P6, P8, P9	
Known-major projections	P1; P5	P5's projection-first ordering after a heartbeat failure.
Type-invention ladder	P1, P2, P6, P9	
Dictionary identity, two instance modes	P1, P3, P5, P6, P8	
Constructive lane with Kripke certificate	P1, P4, P6, P8; Forge	Forge's extension round built the checker.
Dependent elimination with provenance of removed branches	P3, P4, P6, P8, P9	P4 records kernel / exact-fragment / heuristic removal.
Equality repair and transport	P3, P4, P8	
Provider index, tiers	P4, P5, P8, P9	P1's synthesis lemmas; P9's 50-provider cap remark.
Component recipes and the join	P6, P7	P7's context slices.
Providers with contract lemmas	P3, P5, P8, P9	
Carrier grammar and demand rules	P8; P1, P4, P5, P7, P9	P5 adds behavioral generalization.
Recursion routes and schemas	P2, P3, P4, P5, P8, P9	P4's recursion descriptor; P9's <code>RecSketch</code> .
Recursive-call capabilities	all	P2's failed run as motivation.
Motive by dependency-closed generalization; failure-directed generalization	P2, P6	
Proof-carrying motives	all	"Certificate algebras" (P5), "refinement motives" (P1), "packed motives" (P3), "certified motive" (P8).
Accumulator invariants as sub-synthesis	P2, P3, P4, P6, P8, P9	
Measure grammar	P1, P3, P5, P8	
Two products per recursion plan	P4, P6, P8, P9	P6's lowering algorithm from the plan.
Contract decomposer with reconstruction terms	P2, P6, P8, P9	P2's necessary / sufficient / equivalent tags.
Typed observations, residual evaluation, hole consistency table	P2, P3, P6, P7	
Proof-service portfolio and adapter	P1, P3, P5, P6, P9; Forge	Forge's frontend is the reference implementation of the adapter.
Certified CEGIS with unknown branch	P1, P4, P5, P6, P7, P8, P9	P8's finite bound; P7's proof-carrying certifier.
Reversible observational buckets	P1, P2, P3, P5, P9	Three independent ablations show permanent merging loses solutions.
Sound sketch pruning and domain plugin protocol	P3, P5, P6, P8, P9; Forge	P4's exact affine separator family; P3's <code>DomainReply</code> .
SMT with three uses; typed sketches	P2, P5, P6	P5's branch-expanded versus native-first encodings.

Idea	Adopted from	Alternatives and notes
Cache keys and qualified negative entries	P1, P2, P4, P5, P7, P9	
Acceptance gate pipeline	all	Order and steps merged; P8’s exact-statement matching and source re-elaboration; P2’s declaration bundle.
<code>skipKernelTC</code> and <code>async addDecl</code> pitfalls	Forge	Not present in any of the nine proposals.
Receipts	P3, P7, P8	
Isolation profiles	P3, P6, P9	
Frontends and worker/controller	P1, P2, P6, P7, P9	P9 puts recursive bundles at command level.
Compatibility REPL	this document	Required by Decisions 1.1 and 1.2.
Evaluation corpus, leakage rules, adversarial suite	P2, P4, P5, P6, P7, P8, P9	
Gated roadmap	all	Reordered so that the baseline release precedes recursion discovery.

Ideas present in the proposals and *not* adopted: a monolithic **Subtype** answer type (P4’s experiments); persistent native metavariable states across sessions (rejected by P6); permanent observational quotienting outside known finite complete domains (rejected by all); a dedicated learned ranker in the first release (deferred by all); a Haskell-hosted solver process as part of the semantic engine (rejected by all).

B The baseline corpus

Counts per transcript in `C:\Leant\test` at revision `6bf05ad`. “Queries” is the number of `:synth` lines; “where” the number with a behavioral assertion; “cand.” the number of queries whose golden shows at least one candidate; “uninh.” the “provably uninhabited” verdicts; “ctrl” the negative controls (bounded miss, none passed, preflight error).

Transcript	queries	where	cand.	uninh.	ctrl	Features exercised
synth-basic	8	0	7	1	0	products, sums, S/K combinators, Peirce (classical), propositional
synth-behavior	13	12	9	0	4	assertions; <code>where False</code> ; ill-typed and unknown-name controls; verification-window controls
synth-both-frontier	1	0	1	0	0	engine interleaving (ignored)
synth-church-layered-providers	6	0	6	0	0	Church encodings via session providers
synth-church-providers	6	0	6	0	0	Church encodings via providers
synth-church-rankn	69	0	69	0	0	nested polymorphic callbacks, Type 1 families, strict implicits, seeded compositions
synth-church	9	0	9	0	0	Church numerals, pairs, lists
synth-djinn-providers	11	0	11	0	0	session axioms as providers
synth-five-binder-rankn	6	0	6	0	0	five-binder rank-N

Transcript	queries	where	cand.	uninh.	ctrl	Features exercised
synth-inductives	13	0	11	2	0	Bool , Ordering , Option , Except , Decidable , session inductives and structures
synth-instance-implicit	4	0	4	0	0	instance-implicit binders
synth-library	7	0	7	0	0	List.map , foldr , foldl , flatten , append , replicate
synth-manual	31	0	31	0	0	the manual’s tour: state passing, CPS , Option bind, Decidable combinators, classical logic, session opaques
synth-parametric-rankn	9	0	9	0	0	parametric rank-N
synth-prove	5	0	4	1	0	:synth inside :prove ; forall p, p or not p reported uninhabited (fragment)
synth-provider-argument-rankn	2	0	2	0	0	rank-N provider arguments
synth-provider-bound-context-head	3	0	3	0	0	bound context heads
synth-provider-constraint-closure	6	0	6	0	0	instance-constraint closure
synth-provider-contextual-assignment	9	0	6	0	3	contextual assignment; three bounded misses under restricted settings
synth-provider-higher-kind-assignment	12	0	12	0	0	higher-kinded provider assignment
synth-provider-implicit-visible-result	8	0	8	0	0	implicit arguments with visible results
synth-provider-metadata-fitting	2	0	2	0	0	provider metadata
synth-provider-refutation-fallback	5	0	4	1	0	refutation retained while providers run; empty inductive
synth-provider-structural-assignment	6	0	6	0	0	structural provider assignment
synth-quantified-provider	9	0	9	0	0	quantified providers
synth-quartic-rankn	3	0	3	0	0	quartic rank-N
synth-query-closed-rankn	3	0	3	0	0	closed polytypes at local providers
synth-quintic-rankn	2	0	2	0	0	quintic rank-N
synth-recursive-rankn	4	0	4	0	0	recursive inductive families at Type 1
synth-six-binder-rankn	6	0	6	0	0	six-binder rank-N
Total	278	12	266	5	7	

The five “provably uninhabited” queries are **forall a b : Type, a -> b**; **forall a b : Type, Option a -> b**; **forall a b : Type, Pair a b -> Empty**; an empty session inductive under a provider whose instance does not apply; and **forall p : Prop, p or not p**. The first four admit a Lean refutation by instantiating with **Unit** and **Empty**, and by **cases** on the empty inductive; the fifth admits only a fragment certificate under the constructive lane, and a labeled classical candidate under the standard profile.

C Interface sketches

The following declarations fix the shape of the main interfaces. They are design sketches consistent with the prototypes' tested API surface, not compiled code.

```
-- Roles guide scheduling; they never affect typing.
inductive Role | program | typeArg | universe | instance | indexEq | contract |
  termination | invariant | reconstruction

structure Obligation where
  goal      : MVarId
  role      : Role
  residual  : Option Expr          -- residual contract on this hole
  deps      : Array MVarId        -- metavariables whose assignment may change this
                                   obligation

inductive StepResult
| progress (changed : Array MVarId) (new : Array Obligation)
| solved   (term : Expr) (manifest : Dependencies)
| blocked  (on : Array MVarId) (reason : String)
| yielded  (cursor : Cursor) (charged : Work)
| rejected (kind : RejectKind) (cert : Option Checked)
| unsupported (feature : String)
| interrupted (reason : Interrupt)

structure Lane where
  name : Name
  step : Origin -> Task -> Quantum -> WorkerM StepResult

inductive ProofReply
| proof (p : Expr) (patch : Option AssignmentPatch) -- patch only in joint mode
| refutation (q : Expr)
| reduced (goal : MVarId) (reconstruct : Expr)
| unsupported
| unknown (spent : Work)

inductive DomainReply
| unsupported
| rankingHint (score : Int) -- no logical authority
| necessaryConstraint (c : Expr) (necessity : Expr)
| counterexample (input : Expr) (valid : Expr) (violation : Expr)
| noCompletion (cert : Checked) (scope : GrammarId)

-- Only Leant2.Accept can construct this.
structure Accepted where
  private mk ::
  program   : Expr
  proof     : Expr
  receipt   : Receipt
```

The Forge contract adopted for every certified domain checker is:

```
structure CertificateSpec (Problem Certificate : Type) where
  denotes : Problem -> Prop
  check   : Problem -> Certificate -> Bool
  sound   : forall (p : Problem) (c : Certificate), check p c = true -> denotes p
```


References

- [1] V. Reshetnikov et al. *Leant: a Djex-based synthesis REPL for Lean 4*. Repository snapshot 6bf05ad78c467989e68290f2d08bbcd40802d485, September 14, 2026. Files consulted: README.md, docs/synth-internals.md, docs/behavioral-synthesis.md, docs/SYNTHESIS_PROPOSAL.md, src/Leant/Synth/Fragment.hs, test/synth-*.txt, test/synth-*.golden, test/run-tests.sh, test-church/behavior-ledger.md, test-recursive/README.md, test-behavioral/README.md, test-context/global-methods/README.md.
- [2] V. Reshetnikov et al. *Djex*. Standalone snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5; submodule revision pinned by Leant e237e8667190aff38faaa590ce5b12afbfffac452. Files consulted: README.md, docs/architecture.md, docs/rank-n.md, docs/source-typed-evidence-graph.md.
- [3] Leant project. *Rewriting Leant and Djex in Lean 4: Feasibility, Architectural Consequences, Tradeoffs, and Recommended End State*. docs/Leant_Djex_Lean4_Rewrite_Analysis/, August 2026.
- [4] V. Reshetnikov et al. *Forge: a certificate-producing proof-planning layer above Lean's grind*. Repository snapshot 44d191e. Files consulted: README.md, lean/Forge/Frontend.lean, lean/Forge/Checker/, lean/Forge/Design/Contracts.lean, docs/IDEAS-FROM-LEANT-DJEX.md, docs/STATUS.md.
- [5] Proposal 01, *Leant2: A Lean-Native Architecture for Proof-Carrying Program Synthesis*, docs/proposals/01-lean-native-synthesis-architecture/.
- [6] Proposal 02, *Leant2: A Lean-Native Architecture for Type- and Specification-Directed Synthesis*, docs/proposals/02-architecture-checked-experiments/.
- [7] Proposal 03, *Leant2: Lean-Native Synthesis of Programs and Their Contracts*, docs/proposals/03-programs-and-contracts/.
- [8] Proposal 04, *Leant2: Native Dependent Program Synthesis in Lean*, docs/proposals/04-native-dependent-synthesis/.
- [9] Proposal 05, *Leant2: Native Dependent-Type and Behavioral Program Synthesis in Lean*, docs/proposals/05-dependent-behavioral-synthesis/.
- [10] Proposal 06, *Leant2: Lean-Native Program Synthesis with Dependent Types and Behavioral Contracts*, docs/proposals/06-algorithms-checked-pilot/.
- [11] Proposal 07, *Leant2: Lean-Native Program Synthesis from Types and Behavioral Specifications*, docs/proposals/07-architecture-prototypes/.
- [12] Proposal 08, *Leant2: Lean-Native, Contract-Directed Program Synthesis*, docs/proposals/08-architecture-checked-prototypes/.
- [13] Proposal 09, *Leant2: Native Dependent Program Synthesis in Lean (design)*, docs/proposals/09-native-dependent-design/.
- [14] Lean development team. *The Lean Language Reference*, version 4.34.0: The Type System; Universes; Inductive Types; Recursive Definitions; Instance Synthesis; Elaboration and Compilation; Validating a Lean Proof; the grind tactic.
- [15] Lean community. *Metaprogramming in Lean 4*, chapter on MetaM.
- [16] J. Limperg and A. Halkjær From. *Aesop: White-Box Best-First Proof Search for Lean*. CPP 2023.
- [17] C. Norman and J. Avigad. *Canonical for Automated Theorem Proving in Lean*. arXiv:2504.06239, 2025.
- [18] N. Polikarpova, I. Kuraj, A. Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016.

- [19] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala, N. Polikarpova. *Program Synthesis by Type-Guided Abstraction Refinement*. POPL 2020.
- [20] P.-M. Osera and S. Zdancewic. *Type-and-Example-Directed Program Synthesis*. PLDI 2015.
- [21] J. Lubin, N. Collins, C. Omar, R. Chugh. *Program Sketching with Live Bidirectional Evaluation*. ICFP 2020.
- [22] N. Mulleners, J. Jeuring, B. Heeren. *Program Synthesis Using Example Propagation*. PADL 2023.
- [23] T. Knoth, D. Wang, N. Polikarpova, J. Hoffmann. *Resource-Guided Program Synthesis*. PLDI 2019.
- [24] A. Mohamed et al. *Lean-SMT: An SMT tactic for discharging proof goals in Lean*.
- [25] M. Carneiro. *Lean4Lean: Verifying a Typechecker for Lean, in Lean*.
- [26] R. Dyckhoff. *Contraction-Free Sequent Calculi for Intuitionistic Logic*. JSL 57(3), 1992.
- [27] Axiom Math. *AXLE check tool documentation*. <https://axle.axiommath.ai/v1/docs/tools/check/>.